

R.M.K GROUP OF ENGINEERING INSTITUTIONS



R.M.K GROUP OF INSTITUTIONS





Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.



22AM904/22AI903

TEXT AND SPEECH AND SPEECH ANALYTICS

Department: AIML & AIDS
Batch/Year:2023-2027

Created by:

1. Dr. V.VIJAYARAJA,PROFESSOR/AIDS,RMKCET
2. Dr.BADI ALEKHYA,ASP/AIML,RMDEC
- 3.Dr.A.PONMALAR,ASP/RMKEC

Date:15.12.2025



R.M.K.
GROUP OF
INSTITUTIONS



1.TABLE OF CONTENTS

S.NO.	CONTENTS	SLIDE NO.
1	CONTENTS	5
2	COURSE OBJECTIVES	7
3	PRE REQUISITES (COURSE NAMES WITH CODE)	8
4	SYLLABUS (WITH SUBJECT CODE, NAME, LTPC DETAILS)	9
5	COURSE OUTCOMES (6)	10
6	CO- PO/PSO MAPPING	11
7	LECTURE PLAN –UNIT 1	13
8	ACTIVITY BASED LEARNING –UNIT 1	15
9	LECTURE NOTES – UNIT 1	16
10	ASSIGNMENT 1- UNIT 1	51
11	PART A Q & A (WITH K LEVEL AND CO) UNIT 1	52
12	PART B Q s (WITH K LEVEL AND CO) UNIT 1	54
13	SUPPORTIVE ONLINE CERTIFICATION COURSES UNIT 1	56
14	REAL TIME APPLICATIONS IN DAY TO DAY LIFE AND TO INDUSTRY UNIT 1	57
15	ASSESSMENT SCHEDULE	58

S.NO.	CONTENTS	SLIDE NO.
16	PRESCRIBED TEXT BOOKS & REFERENCE BOOKS	59
17	MINI PROJECT SUGGESTIONS	60

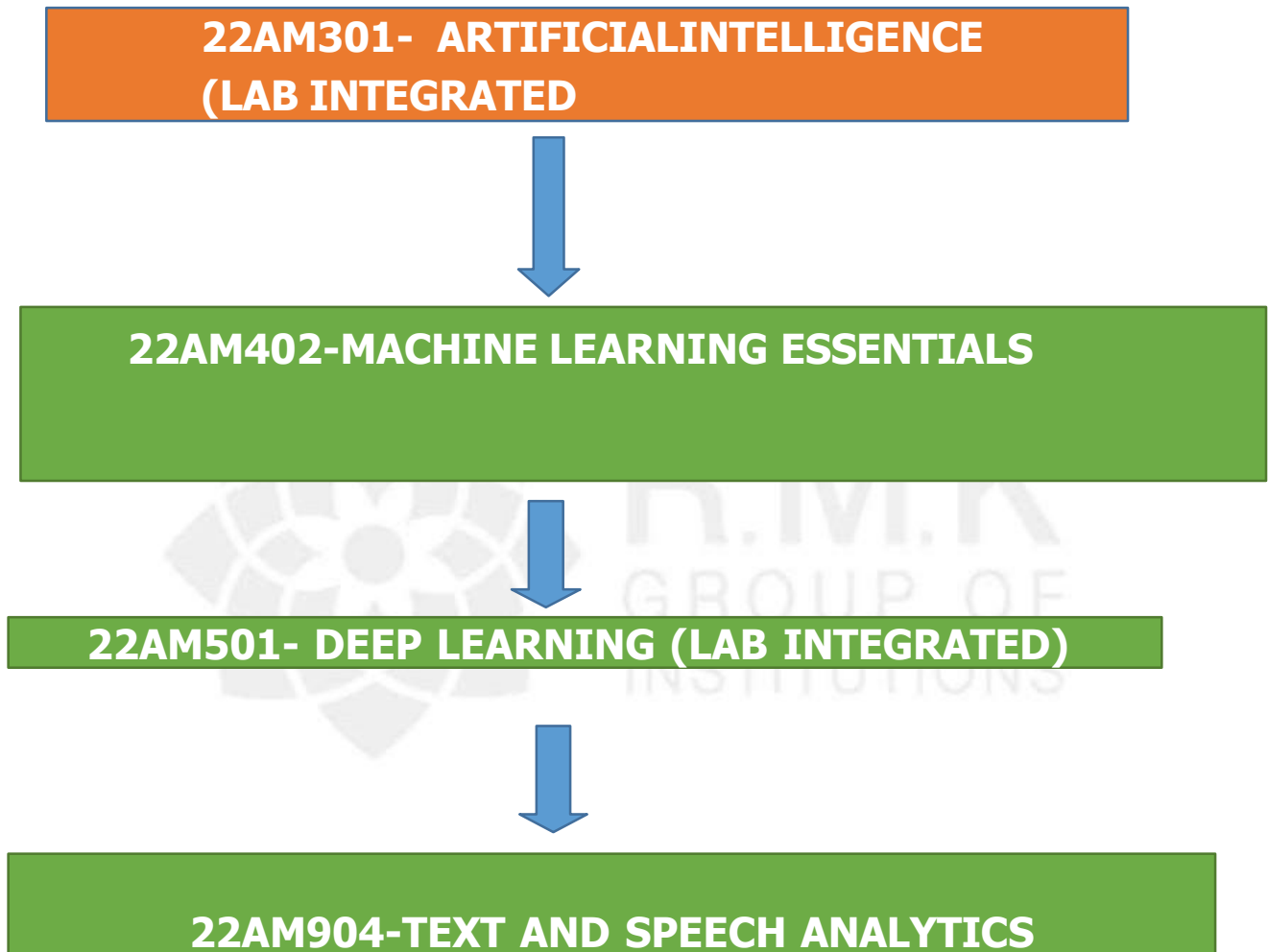


2. COURSE OBJECTIVES

- To introduce the tools and techniques for performing text and speech analytics in diverse contexts.
- To understand the tools and technologies involved in developing text and speech applications.
- To demonstrate the use of computing for building applications in text and speech processing.
- To use information Retrieval Techniques to build and evaluate text processing systems.
- To apply advanced speech recognition methodologies in practical applications.

3. PRE REQUISITES

✿ PRE-REQUISITE CHART



4.SYLLABUS

22AM904-TEXT AND SPEECH ANALYTICS

L T P C
3 0 0 3

UNIT I TEXT PROCESSING

9

Speech and Language Processing - Regular Expression - Text normalization – Edit Distance - Lemmatization – Stemming – N-gram Language Models - Vector Semantics and Embeddings.

UNIT II TEXT CLASSIFICATION

9

Text Classification Tasks – Language Model – Neural Language Models – RNNs as Language Models – Transformers and Large Language Models.

UNIT III QUESTION ANSWERING AND DIALOGUE SYSTEMS

9

Information Retrieval – Dense Vectors – Neural IR for Question Answering – Evaluating Retrieval-based Question Answering – Frame-based Dialogue Systems – Dialogue Acts and Dialogue State – Chatbots – Dialogue System Design.

UNIT IV TEXT TO SPEECH SYNTHESIS

9

Automatic Speech Recognition Task – Feature Extraction for ASR: Log Mel Spectrum – Speech Recognition Architecture – CTC - ASR Evaluation: Word Error Rate – TTS – Speech Tasks.

UNIT V SPEECH RECOGNITION

9

LPC for speech recognition - Hidden Markov Model (HMM) - Training procedure for HMM- subword unit model based on HMM - Language models for large vocabulary speech recognition - Overall recognition system based on subword units - Context dependent subword units- Semantic post processor for speech recognition



5.COURSE OUTCOME

Course Code	Course Outcome Statement	Cognitive / Affective Level of the Course Outcome	Course Outcome
Course Outcome Statements in Cognitive Domain			
22AM904	Apply the fundamental techniques in text processing for various NLP tasks.	Apply K3	CO1
22AM904	Implement advanced language models and improve text classification accuracy.	Apply K3	CO2
22AM904	Designing text processing systems using state-of-the-art techniques	Apply K3	CO3
22AM904	Design, implement, and evaluate ASR and TTS systems.	Apply K4	CO4
22AM904	Apply advanced speech recognition methodologies in practical applications	Apply K4	CO5
22AM904	Use information Retrieval Techniques to build and evaluate text processing systems	Apply K4	CO5

6.CO-PO/PSO MAPPING

Correlation Matrix of the Course Outcomes to Programme Outcomes and Programme Specific Outcomes .

Course Outcomes (Cos)		Programme Outcomes (POs), Programme Specific Outcomes (PSOs)														
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
22AM904.1	K 2	3	3	1	-	-	-	-	-	-	-	-	-	-	-	-
22AM904.2	K 3	3	2	1	-	-	-	-	-	-	-	-	-	-	-	-
22AM904.3	K 3	3	2	1	-	-	-	-	-	-	-	-	-	-	-	-
22AM904.4	K 3	3	3	2	-	-	-	-	-	-	-	-	-	-	-	-
21AM904.5	K 2	3	2	2	-	-	-	-	-	-	-	-	-	-	-	-
21AM904.6	K 2	3	2	2												



UNIT I



LECTURE PLAN – UNIT I

UNIT I INTRODUCTION							
Sl. No	TOPIC	NO OF PERIODS	PROPOSED LECTURE	ACTUAL LECTURE	PERTAINING CO(s)	TAXONOMY LEVEL	MODE OF DELIVERY
			PERIOD	PERIOD			
1	Speech and Language Processing	1			CO1	K2	MD1, MD5
2	Regular Expression	1			CO1	K1	MD1, MD5
3	Regular Expression	1			CO1	K2	MD1, MD5
4	Text normalization	1			CO1	K2	MD1, MD5
5	Edit Distance	1			CO1	K2	MD1, MD5
6	Lemmatization	1			CO1	K2	MD1, MD5
7	Stemming	1			CO1	K5	MD1, MD5
8	N-gram Language Models	1			CO1	K2	MD1, MD5
9	Vector Semantics and Embeddings	1			CO1	K5	MD1, MD5

LECTURE PLAN – UNIT I

TEXT PROCESSING

❁ ASSESSMENT COMPONENTS

- ❁ AC 1. Unit Test
- ❁ AC 2. Assignment
- ❁ AC 3. Course Seminar
- ❁ AC 4. Course Quiz
- ❁ AC 5. Case Study
- ❁ AC 6. Record Work
- ❁ AC 7. Lab / Mini Project
- ❁ AC 8. Lab Model Exam
- ❁ AC 9. Project Review

MODE OF DELEIVERY

- MD 1. Oral presentation
- MD 2. Tutorial
- MD 3. Seminar
- MD 4 Hands On
- MD 5. Videos
- MD 6. Field Visit



R.M.K.
GROUP OF
INSTITUTIONS



8. ACTIVITY BASED LEARNING : UNIT – I

ACTIVITY 1:

Regular Expressions:

Write regex patterns to:

- Extract all email addresses from a given text file.
- Identify sentences containing numerical values.
- Remove punctuation from the text.
- Use Python's re library for implementation.

Text Normalization:

- Convert all text into lowercase.
- Remove stopwords (use NLTK or SpaCy's stopword list).

Stemming and Lemmatization:

- Perform stemming using Porter Stemmer or Snowball Stemmer.
- Perform lemmatization using WordNetLemmatizer in NLTK.
- Compare outputs of stemming and lemmatization on sample sentences.

9. LECTURE NOTES : UNIT – I

UNIT I TEXT PROCESSING

Speech and Language Processing - Regular Expression - Text normalization – Edit Distance
Lemmatization – Stemming – N-gram Language Models - Vector Semantics and
Embeddings



UNIT 1 TEXT PROCESSING

Speech and Language Processing

Speech and Language Processing involves the development of techniques and systems that allow computers to understand, interpret, and generate human languages (both spoken and written). It encompasses multiple domains of research and applications, such as speech recognition, natural language processing (NLP), and text-to-speech synthesis. The field is inherently interdisciplinary, combining linguistics, computer science, and artificial intelligence (AI).

Key Components of Speech and Language Processing:

1. Speech Recognition (Automatic Speech Recognition - ASR):

- The process of converting spoken words into text. ASR systems are used in voice assistants like Siri, Google Assistant, and Alexa.
- Example: When you say, “Hey Siri, what’s the weather today?” the voice is converted into text by an ASR system, which is then processed to generate a response.

2. Natural Language Processing (NLP):

- NLP deals with the interaction between computers and human language. It enables machines to process and understand human language, making it possible to interpret commands, perform translations, and extract meaningful information.
- Example: NLP is used in chatbots, which understand user queries and provide appropriate responses.

3. Text-to-Speech (TTS):

- Converts written text into spoken words. It is the reverse of ASR and is commonly used in applications for people with disabilities or in virtual assistants.
- Example: If you type a message in an app like Google Translate, it can read out the text in various languages using TTS technology.

3. Speech Synthesis:

- Involves the creation of human-like speech from text. It uses algorithms to mimic the natural flow and intonation of speech.
- Example: Google’s Text-to-Speech service, which reads out the content on your smartphone screen aloud.



- This process involves converting raw text into a standard format by removing punctuation, converting text to lowercase, correcting misspelled words, etc.
- Example: “I can't” is normalized to “I cannot”.

3.Edit Distance:

- Edit Distance measures the number of operations required to convert one string into another. This is often used in speech recognition systems to measure how close a spoken word is to the expected transcription.
- Example: In ASR, if “hello” is misheard as “hellos”, the edit distance between the two words is 1 (since one letter is added).

4.Lemmatization and Stemming:

- Stemming is a technique used to reduce words to their root form (e.g., “running” → “run”).
- Lemmatization involves reducing words to their base form, considering its meaning (e.g., “better” → “good”).
- Example: In text processing, lemmatization helps in analyzing text data by reducing similar words to one form for consistency.

5.N-gram Language Models:

- N-grams are sequences of 'n' words used to predict the next word or sequence in a sentence. They are fundamental for language modeling and are widely used in both speech recognition and NLP tasks.
- Example: In a bigram model (2-grams), the sequence "I am" can predict the next word "going" in the phrase "I am going to the store".

6.Vector Semantics and Embeddings:

- Vector semantics involves representing words or phrases as vectors in a multi- dimensional space. This is crucial for deep learning models in NLP.
- Example: Word embeddings like Word2Vec or GloVe convert words into vector representations, making it easier to compute similarities between words or find relationships (e.g., "king" - "man" + "woman" $\approx$ "queen").



Applications of Speech and Language Processing

1. Voice Assistants (e.g., Siri, Alexa, Google Assistant):

- These systems rely heavily on speech recognition and NLP to interpret voice commands and generate spoken responses.
- **Example:** A user asks, “What’s the capital of France?” The system recognizes the speech, processes it using NLP techniques, and replies, “Paris.”

2. Speech-to-Text Systems:

- Used in transcription services and automated subtitles. ASR systems are widely used in transcription tools like Otter.ai.
- **Example:** In medical or legal transcription, ASR systems convert doctor-patient or lawyer-client interactions into written reports.

3. Chatbots and Virtual Assistants:

- These systems engage in text-based conversations with users, using NLP to understand queries and generate responses.
- **Example:** A customer service chatbot uses NLP to understand and respond to inquiries about account balance or technical support.

4. Language Translation Systems:

- Systems like Google Translate use NLP techniques to translate text from one language to another.

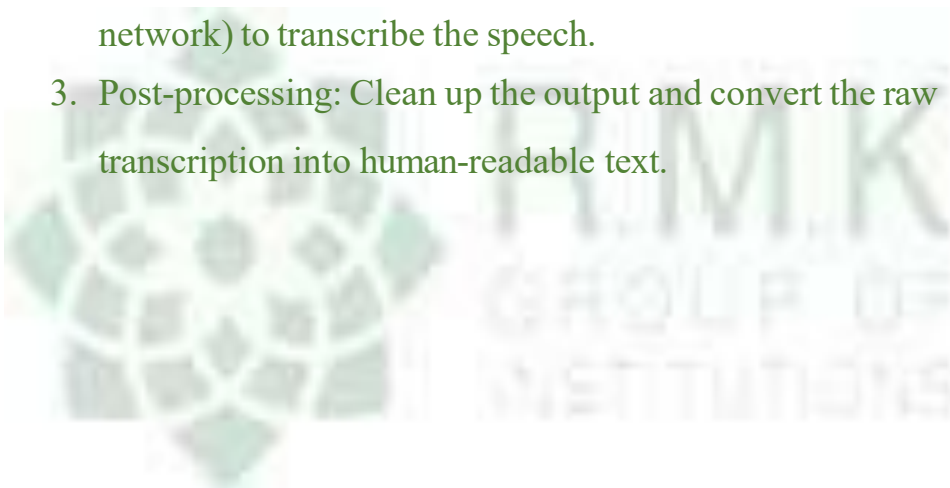
- **Example:** Translating the phrase "Hola, ¿cómo estás?" from Spanish to "Hello, how are you?" in English.

Example: A Simple Speech Recognition Task

Problem: Convert the sentence "I am learning speech processing" into text using ASR.

•Steps:

1. Feature Extraction: Convert audio to a form that can be processed by a model, such as a Mel spectrogram.
2. Model Processing: Use a pre-trained model (like a deep neural network) to transcribe the speech.
3. Post-processing: Clean up the output and convert the raw transcription into human-readable text.



Regular Expressions (Regex)

A Regular Expression (Regex) is a sequence of characters that forms a search pattern. It is mainly used for pattern matching with strings, or for searching and manipulating text. Regular expressions are essential in tasks such as text searching, text extraction, and data cleaning, particularly in speech and language processing, where preprocessing text or speech transcriptions is often required.

Key Concepts of Regular Expressions:

1. Literal Characters:

- Basic characters that match themselves in a string.
- Example: The regex `apple` matches the string "apple".

2. Meta-characters: These are special characters that have specific meanings. Commonly used meta-characters include:

- `.` (dot): Matches any single character (except newline).
- `^`: Anchors the match at the beginning of the string.
- `$`: Anchors the match at the end of the string.
- `*`: Matches zero or more of the preceding character.
- `+`: Matches one or more of the preceding character.
- `?`: Matches zero or one of the preceding character.

1. Character Classes: A character class defines a set of characters that can match a position in the string.

Example:

- `[a-z]`: Matches any lowercase letter.
- `[A-Z]`: Matches any uppercase letter.
- `[0-9]`: Matches any digit.
- `[aeiou]`: Matches any vowel.

2. Predefined Character Classes:

- `\d`: Matches any digit (equivalent to `[0-9]`).
- `\w`: Matches any word character (alphanumeric + underscore).
- `\s`: Matches any whitespace character (spaces, tabs, line breaks).



`\b`: Matches a word boundary.

5. Quantifiers:

Quantifiers specify the number of occurrences of a character or group to match.

Example:

`a{3}`: Matches exactly three 'a's in a row (e.g., "aaa").

`a{2,4}`: Matches between 2 and 4 'a's in a row (e.g., "aa", "aaa", "aaaa").

6. Grouping and Capturing:

Parentheses `()` are used to group parts of the regular expression, allowing you to apply operators to entire sections of the pattern.

Example:

- `(abc)+`: Matches one or more occurrences of "abc".

Capturing groups store the matched text, which can be referenced later.

7. Alternation:

The pipe symbol `|` represents an "OR" operation in regular expressions.

Example:

`apple|banana`: Matches either "apple" or "banana".

8. Escape Sequences:

Some characters are reserved in regex (e.g., `.` or `*`). To use these characters as literals, they must be escaped with a backslash `\`.

Example:

- `\.`: Matches the literal dot character, not any character.

Applications of Regular Expressions in Speech and Language Processing

1. Text Normalization:

Regular expressions are widely used to clean and preprocess raw text before feeding it into text processing models.

Example:

Removing unwanted punctuation or special characters from text.

Transforming all characters to lowercase for uniformity.

Example Regex: To remove punctuation from text:

regex Copy code `[^\w\s]`

This regex matches any character that is not a word character or whitespace.

2. Tokenization:

Breaking down sentences into tokens (words, punctuation, etc.) is a fundamental step in text processing. Regular expressions help segment the text into individual words and phrases.

Example:

Splitting text into words based on spaces and punctuation.

Regex for splitting sentences into words: `\w+`

3 Pattern Matching and Extraction:

Regex is often used to search for specific patterns in text, such as email addresses, dates, phone numbers, or specific keywords.

Example:

- Extracting phone numbers from a document:

regex Copy code

```
\d{3}-\d{3}-\d{4}
```

This regex matches a phone number in the format xxx-xxx-xxxx.

4. Named Entity Recognition (NER):

In text processing, regex is used to identify entities such as names, dates, and places by matching predefined patterns.

Example:

Matching dates: `\d{2}/\d{2}/\d{4}` (matches "12/05/2022").

5. Speech-to-Text Transcription Cleanup:

After speech recognition transcribes audio into text, regular expressions can be used to remove errors like extra spaces, incomplete words, or unwanted symbols.

Example: Removing extra spaces after transcription:

regex Copy code

```
\s{2,}
```

Example of Regular Expression Usage:

This matches two or more consecutive spaces and replaces them with a single space.

Let's say you want to extract dates in the format DD/MM/YYYY from a text. You could use the following regex:

regex Copy code

```
\d{2}/\d{2}/\d{4}
```

Explanation:

- `\d{2}`: Matches two digits (for the day).
- `/`: Matches the slash separator.
- `\d{2}`: Matches two digits (for the month).
- `/`: Matches the slash separator.
- `\d{4}`: Matches four digits (for the year).

Example Match:

- Input text: "The event is scheduled for 12/05/2022, and another one on 15/06/2023."
- Output: "12/05/2022" and "15/06/2023" will be matched and extracted.



Text Normalization

Text Normalization is a crucial preprocessing step in natural language processing (NLP) and speech analytics. It involves transforming text into a standard, consistent format to remove any inconsistencies or variations that may exist in the raw input data. The goal of text normalization is to prepare the text for further processing tasks such as tokenization, part-of-speech tagging, and named entity recognition (NER).

Key Steps in Text Normalization

Lowercasing:

Converts all characters in the text to lowercase, ensuring uniformity and eliminating case-based differences.

Why It's Needed: In many NLP tasks, "Apple" and "apple" should be treated as the same word.

Example:

Input: "I love the new Apple products."

Output: "i love the new apple products."

Removing Punctuation:

Punctuation marks (e.g., commas, periods, exclamation marks) are often removed unless they are part of the linguistic structure (e.g., sentence segmentation).

Why It's Needed: Punctuation may not carry significant meaning for some tasks like word frequency analysis.

Example:

Input: "Hello, world!"

Output: "Hello world"

Removing Special Characters:

Special characters such as @, #, or & are removed from the text unless they hold specific importance in certain contexts (e.g., hashtags in social media posts).

Why It's Needed: Special characters can interfere with the analysis of the core textual content.

Example:

Input: “#DataScience is awesome!”

Output: “DataScience is awesome”

Expanding Contractions:

Contractions (e.g., "I'm", "don't", "can't") are expanded into their full forms to reduce ambiguity.

Why It's Needed: Expanding contractions ensures that words are processed in their standard form.

Example:

Input: “I can’t believe it!”

Output: “I cannot believe it!”

Removing Stop Words:

Stop words (e.g., “the”, “a”, “and”, “in”) are often removed because they carry little semantic meaning in most NLP tasks.

Why It's Needed: Removing stop words can help improve computational efficiency and focus on the most important terms in the text.

Example:

Input: “The quick brown fox jumps over the lazy dog.”

Output: “quick brown fox jumps lazy dog”

Stemming and Lemmatization:

Stemming involves reducing words to their root form by chopping off suffixes (e.g., "running" becomes "run").

Lemmatization considers the meaning of the word and reduces it to its base form (e.g., “better” becomes “good”).

Why It's Needed: These techniques help in reducing variations of the same word, making it easier to analyze.

Example:

Input: “running” (stemming) → Output: “run”

Input: “better” (lemmatization) → Output: “good”



Spelling Correction:

Correcting misspelled words is a part of text normalization, ensuring that text is properly standardized for analysis.

Why It's Needed: Misspelled words can lead to misinterpretation and inaccurate results in NLP tasks.

Example:

Input: "I love progamming."

Output: "I love programming."

Handling Numerals:

Text normalization may also involve handling numbers by either converting them to words or removing them, depending on the application.

Why It's Needed: Numbers can sometimes be irrelevant in text processing tasks, especially in tasks like sentiment analysis.

Example:

Input: "I have 3 apples."

Output: "I have three apples" (if converting numbers to words) or "I have apples" (if removing numbers).

Applications of Text Normalization

1. Speech-to-Text Processing:

In speech recognition, text normalization is essential to convert transcribed speech into a clean and readable format.

Example: Converting "I'm lovin' it!" into "I am loving it."

2. Sentiment Analysis:

Normalizing text ensures that stop words, punctuation, and contractions do not interfere with sentiment classification algorithms.

○Example: "This is soooo good!" becomes "This is so good."

3. Information Retrieval (IR):

When processing documents or queries, text normalization helps in improving the retrieval of relevant results by ensuring consistency across the corpus.

Example: "Natural Language Processing" and "natural language processing" should be treated as the same term.

4. Machine Translation:

Normalizing text ensures that the translation model processes text consistently, handling contractions, punctuation, and other variations.

Example: "I'm going to the store" becomes "I am going to the store" before translation.

4. Named Entity Recognition (NER):

In NER, normalization is used to standardize entity names, such as converting

“Dr. Smith” into “Doctor Smith” to maintain consistency.

Example: "Mr. John Doe" becomes "Mr. John Doe" (Standardized for name matching).

Example of Text Normalization:

Problem: Normalize the following text:

Input Text: “I’m lovin' the new iPhone 12!!! It's amaZING.”

Normalization Process:

- 1.Convert to lowercase: "i'm lovin' the new iphone 12!!! it's amazing."
- 2.Remove punctuation: "i'm lovin the new iphone 12 its amazing"
- 3.Expand contractions: "i am loving the new iphone 12 its amazing"
- 4.Correct spelling: "I am loving the new iPhone 12 its amazing"

Output: "I am loving the new iPhone 12 its amazing"

Edit Distance

Edit Distance is a measure of the difference between two strings (e.g., words or sequences of text). It quantifies how many basic operations (insertions, deletions, substitutions) are needed to transform one string into another. Edit distance is a fundamental concept in text processing, especially in tasks like spell checking, text correction, machine translation, and speech recognition.

Types of Edit Distance

1. Levenshtein Distance:

Levenshtein Distance is the most commonly used edit distance algorithm. It computes the minimum number of single-character edits required to convert one string into another, where each edit can be one of the following:

- Insertion: Adding a character to a string.
- Deletion: Removing a character from a string.
- Substitution: Replacing one character with another.

• Example:

- String 1: “kitten”
- String 2: “sitting”
- The operations required are:
 1. Substitute 'k' with 's': "kitten" → "sitten"
 2. Insert 'i' at the end: "sitten" → "sittin"
 3. Insert 'g' at the end: "sittin" → "sitting"
- Total distance = 3 (3 operations)

2. Damerau-Levenshtein Distance:

The Damerau-Levenshtein Distance is an extension of the Levenshtein distance that also considers transpositions (swapping two adjacent characters) as a valid operation.

Example:

- String 1: “ab”
- String 2: “ba”
- The Damerau-Levenshtein distance is 1, as only a transposition is required.

3. Hamming Distance:

Hamming Distance is a special case of edit distance that only works on strings of the same length and counts the number of positions at which the corresponding characters are different.

Example:

- String 1: “karolin”
- String 2: “kathrin”
- The Hamming distance is 3 because the characters at positions 3, 5, and 6 differ.

Applications of Edit Distance

1. Spell Checking:

In spell checking, the edit distance between a misspelled word and a dictionary word is calculated. Words with a small edit distance are considered potential corrections.

Example:

For the misspelled word "recieve," the edit distance to "receive" is 1 (substitution of 'i' with 'e').

2. Text Similarity:

Edit distance is used to measure the similarity between two pieces of text. It helps in tasks like document clustering, plagiarism detection, and deduplication.

Example:

If two sentences "I love programming" and "I love prograaming" are compared, the edit distance is 1 (one extra 'a' in the second word).

3. Speech Recognition:

In speech-to-text systems, the output of an ASR system (speech recognition) is compared with the correct transcription to calculate the edit distance, often using Word Error Rate (WER), which is derived from edit distance.

Example:

Output transcription: "I have a cat"

Correct transcription: "I have a hat"

The edit distance is 1 (substitution of 'c' with 'h').

4. Plagiarism Detection:

When checking for plagiarism, edit distance is used to measure how much two pieces of text are altered or how similar they are.

Example: Comparing a student's work with a known text to detect if substantial edits have been made.

5. Machine Translation:

In machine translation, edit distance is used to evaluate how close the translated text is to the human reference translation. It helps in measuring the quality of translation models.

Example: Comparing machine-generated translations with human reference translations.



Computing Edit Distance

The most common method to compute edit distance is using Dynamic Programming. Here's how the Levenshtein Distance can be computed using dynamic programming:

1. Initialize a matrix where the rows represent the characters of the first string, and the columns represent the characters of the second string.
2. Fill the matrix by calculating the minimum number of operations needed to transform substrings of the first string into substrings of the second string.
3. The final value in the bottom-right cell of the matrix gives the minimum edit distance between the two strings.

Example of Levenshtein Distance Calculation

Let's calculate the Levenshtein distance between the words "kitten" and "sitting":

		a = sitting					
		b = kitten					
	#	K	I	T	T	E	N
#	0	1	2	3	4	5	6
S	1	1	2	3	4	5	6
I	2	2	1	2	3	4	5
T	3	3	2	1	2	3	4
T	4	4	3	2	1	2	3
I	5	5	4	3	2	2	3
N	6	6	5	4	3	3	2
G	7	7	6	5	4	4	3

- The distance between "kitten" and "sitting" is 3, as it requires 3 operations (substituting 'k' with 's', inserting 'i' at the end, and inserting 'g' at the end).

Word Error Rate (WER):

WER is a metric used to evaluate the performance of speech-to-text systems. It is calculated as the edit distance between the reference (correct transcription) and the hypothesis (ASR output), divided by the total number of words in the reference.

WER Formula:

1. WER Formula:

$$\text{WER} = \frac{\text{Substitutions} + \text{Insertions} + \text{Deletions}}{\text{Total Words in Reference}}$$

Where:

S is the number of substitutions,

D is the number of deletions,

I is the number of insertions,

N is the total number of words in the reference.

Example:

Reference: "I love programming"

Output: "I have programming"

Substitution (love → have): 1 substitution

WER = $1/3 = 0.33$ or 33% error rate.

Lemmatization

Lemmatization is the process of reducing a word to its base or root form, known as the lemma, while considering the context and meaning of the word. Unlike stemming, which simply removes suffixes without understanding the word's meaning, lemmatization ensures that the resulting word is an actual word in the language.

Lemmatization uses a vocabulary and morphological analysis of words to return their base form.

For example:

The lemma of "running" is "run".

The lemma of "better" is "good" (based on context and meaning).

Key Features of Lemmatization

1.Context Awareness:

Lemmatization considers the meaning and part of speech (POS) of the word.

For example, "flies" as a noun is reduced to "fly," while as a verb, it is also reduced to "fly."

2.Dependency on POS Tagging:

The lemmatizer requires POS tags to determine the correct lemma.

For example, "saw" can be a noun (the tool) or a verb (past tense of "see"). The lemma is determined based on context.

3.Dictionary-Based Approach:

Lemmatization relies on dictionaries or lexicons to determine the base form of a word.

Process of Lemmatization

1.POS Tagging:

The word's part of speech is identified (e.g., noun, verb, adjective).

Example:

Input: "The boys are playing in the park."

POS Tags: [The (DT), boys (NNS), are (VB), playing (VBG), in (IN), the (DT), park (NN)].

Morphological Analysis:

The morphological structure of the word is analyzed to determine its lemma.

Example:

"Playing" → root: "play" (verb).

Lookup in Lemmatization Dictionary:

The lemma is looked up in the lexicon or dictionary based on the POS tag and root form.

Examples of Lemmatization

1.Basic Examples:

Words like "running," "runs," and "ran" → Lemma: "run".

Words like "better" → Lemma: "good" (based on context).

2.Sentence-Level Example:

Input Sentence: "The children were playing in the gardens."

Lemmatized Output: "The child be play in the garden."

3.Ambiguity Example:

Word: "barked"

As a verb (past tense): Lemma → "bark."

As a noun (the sound of a dog): Lemma → "bark."

Applications of Lemmatization

1. Search Engines:

Lemmatization helps improve search results by matching queries to documents, regardless of word variations.

Example: A user searches for "running," and the engine retrieves documents containing "run," "runs," or "ran."

2. Text Classification:

Reducing words to their lemma helps create consistent input for machine learning models.

Example: In sentiment analysis, words like "happy" and "happier" are reduced to "happy," ensuring consistent feature extraction.

3. Speech-to-Text Systems:

Lemmatization ensures that speech transcriptions are converted into meaningful, standardized text for further processing.

Example: Converting "talking" in a transcript to "talk" for language modeling.

4. Machine Translation:

Lemmatization ensures consistency when translating between languages by standardizing word forms.

Example: Translating "jumping" and "jumps" into a consistent word in the target language.

5. Question Answering Systems:

Lemmatization enables systems to understand user queries better by reducing variations in word forms.

Example: A question about "children playing" can match documents containing "child play."

Example: Lemmatization vs. Stemming

1. Input Word: "caring"

Lemmatization: "care"

Stemming: "car"

2. Input Word: "flying"

Lemmatization: "fly"

Stemming: "fli"

3. Input Word: "better"

Stemming: "bet"

Lemmatization: "good"

Stemming

- Stemming is the process of reducing words to their base or root form by removing affixes (prefixes or suffixes). Unlike lemmatization, stemming does not consider the context or meaning of the word. Instead, it applies a set of heuristic rules to trim words down to their "stem."
- Stemming is widely used in text preprocessing tasks for natural language processing (NLP) applications, such as search engines, text classification, and information retrieval.

Key Features of Stemming

1. Rule-Based Approach:

Stemming uses rules to remove common prefixes and suffixes.

Example: Words ending in "ing," "ed," or "ly" are reduced by stripping these endings.

2. Not Context-Aware:

Stemming does not consider the word's meaning or part of speech (POS).

Example: The word "better" is stemmed to "bet," even though "good" is the actual lemma.

3. Produces Non-Words:

Stems are often not valid words in the language.

Example: "Studies" is stemmed to "studi."



Examples of Stemming

Basic Examples:

"Running" → "run" "Studies" → "studi" "Caring" → "car"

Sentence-Level Example:

Input: "The boys are running quickly."

Output: "The boy are run quick."

- Different Word Forms:

- "Connection," "connections," "connected," and "connecting" are all reduced to "connect."

Common Stemming Algorithms

1. Porter Stemmer:

One of the most widely used stemming algorithms.

Applies a series of rules to remove common suffixes.

Example:

Input: "caresses," "flies," "dies"

Output: "caress," "fli," "die"

2. Lancaster Stemmer:

A more aggressive stemming algorithm that produces shorter stems.

Example:

Input: "running"

Output: "run"

3. Snowball Stemmer:

- An improved version of the Porter stemmer, also known as the Porter2 stemmer.
- Supports multiple languages and is less aggressive than the Lancaster stemmer.

4. Regex-Based Stemmer:

- Uses regular expressions to define simple rules for stemming.
- Example: Removing "-ing," "-ed," or "-ly" endings.



Applications of Stemming

1. Search Engines:

- Stemming helps search engines retrieve relevant documents by matching different word forms.
- **Example:** A search for "running" retrieves results containing "run," "runs," or "ran."

2. Text Classification:

- Reducing words to their stems improves the efficiency of text classification models by reducing dimensionality.
- **Example:** In sentiment analysis, "happy" and "happiness" are treated as the same feature.

3. Information Retrieval:

- Stemming enhances the matching of user queries to relevant documents by normalizing word forms.
- **Example:** Searching for "connections" in a database also retrieves documents containing "connected."

4. Spam Detection:

- Stemming reduces variations in word forms, making it easier to detect patterns in spam messages.
- **Example:** "offer," "offered," and "offering" are normalized to "offer."

Comparison: Lemmatization vs. Stemming

Aspect	Lemmatization	Stemming
Output	Produces meaningful words (e.g., "better" → "good").	May produce non-words (e.g., "better" → "bet").
Context Awareness	Considers context and POS.	Ignores context and POS.
Accuracy	High accuracy in identifying root words.	Lower accuracy as it uses simple rules.
Speed	Slower (requires dictionary lookup).	Faster (rule-based).

N-gram Language Models

- An N-gram language model is a statistical language model used to predict the likelihood of a sequence of words or tokens. It divides text into chunks of n words or tokens (N-grams) and estimates the probability of a word based on its preceding n-1 words.
- This approach is foundational in text and speech analytics for tasks like text generation, speech recognition, and machine translation.

Key Concepts of N-grams

Definition:

An N-gram is a contiguous sequence of n items (words, characters, or phonemes) from a given text or speech input.

Examples:

Unigram (n=1): ["I", "love", "NLP"]

Bigram (n=2): ["I love", "love NLP"]

Trigram (n=3): ["I love NLP"]

2. Probability Estimation:

- The probability of a word w_i given its context is calculated as:

$$P(w_i | w_{i-n+1}, \dots, w_{i-1}) = \frac{\text{Count}(w_{i-n+1}, \dots, w_i)}{\text{Count}(w_{i-n+1}, \dots, w_{i-1})}$$

- For example, in a bigram model:

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

3. Markov Assumption:

- N-gram models rely on the Markov assumption, which states that the probability of a word depends only on the preceding n-1 words and not the entire sequence.

Steps to Build an N-gram Model

1. Tokenization:

Split the text into words or tokens.

Example: "I love NLP" → ["I", "love", "NLP"]

2. Generate N-grams:

Extract sequences of n contiguous tokens.

Example for bigrams: ["I love", "love NLP"]

3. Calculate Frequencies:

Count occurrences of each N-gram.

Example:

“ I love learning NLP with N-grams”

“I love learning NLP with N-grams.”

1. Unigram (n=1):

- ["I", "love", "learning", "NLP", "with", "N-grams"]
- Probability:

$$P(\text{"love"}) = \frac{\text{Count}(\text{"love"})}{\text{Total Words}}$$

2. Bigram (n=2):

- ["I love", "love learning", "learning NLP", "NLP with", "with N-grams"]
- Probability:

$$P(\text{"learning"}|\text{"love"}) = \frac{\text{Count}(\text{"love learning"})}{\text{Count}(\text{"love"})}$$

3. Trigram (n=3):

- ["I love learning", "love learning NLP", "learning NLP with", "NLP with N-grams"]
- Probability:

$$P(\text{"NLP"}|\text{"love learning"}) = \frac{\text{Count}(\text{"love learning NLP"})}{\text{Count}(\text{"love learning"})}$$

Applications of N-gram Models

➤ **Speech Recognition:**

Predicts the most likely next word to improve transcription accuracy.

Example: In "I want to", the trigram model predicts "go" or "eat" based on training data.

➤ **Autocomplete and Text Prediction:**

Suggests the next word based on previous inputs.

Example: Typing "How are" suggests "you" in predictive text.

➤ **Spelling Correction:**

Identifies the most likely word in the context of surrounding words.

Example: "Ths is a tst" → "This is a test" using bigram probabilities.

➤ **Machine Translation:**

Helps in aligning and translating phrases by considering word sequences.

Example: "Je t'aime" → "I love you," considering bigrams like "I love."

➤ **Sentiment Analysis:**

Considers word combinations to determine sentiment.

Example: "Very happy" is more positive than "Very sad."

➤ **Language Modeling:**

Predicts the next word in a sequence, commonly used in NLP tasks.

Example: In "The cat sat on the," the model predicts "mat."

Advantages of N-gram Models

1.Simple and Effective:

- Easy to implement and understand.

2.Computationally Efficient:

- Works well for small n values with manageable datasets.

3.Good for Short-Term Dependencies:

- Captures immediate context effectively.



Vector Semantics and Embeddings

Vector Semantics is a method of representing the meaning of words as mathematical vectors in a continuous, high-dimensional space. These vectors capture semantic relationships between words, enabling machines to understand and analyze language more effectively.

Embeddings are the actual vector representations of words, phrases, or sentences. They map discrete linguistic units into a continuous vector space, where similar words are closer to each other.

Key Concepts in Vector Semantics and Embeddings

1. Word Vectors:

Words are represented as points in a multi-dimensional space.

The closer two words are in this space, the more similar their meanings.

2. Context-Based Representations:

Word embeddings are generated based on the contexts in which words appear, capturing semantic and syntactic relationships.

3. Dimensionality Reduction:

Instead of representing words as high-dimensional sparse vectors (e.g., one-hot encoding), embeddings represent them as dense vectors in a smaller dimensional space.

Word Embedding Models

1. Count-Based Models:

Use co-occurrence matrices to represent word relationships.

Example: Latent Semantic Analysis (LSA).

2. Predictive Models:

Predict word embeddings directly by training neural networks.

Examples: Word2Vec, GloVe.

3. Contextual Models:

Capture word meaning based on surrounding context.

Examples: BERT, ELMo.



Popular Word Embedding Techniques

1. Word2Vec:

Developed by Google, Word2Vec creates word embeddings using two methods:

- Skip-Gram: Predicts the context words from a given word.
- CBOW (Continuous Bag of Words): Predicts a target word from its context words.

Example:

▪Input: "The cat sat on the mat."

▪Output: Vectors for words like "cat," "sat," and "mat," where "cat" and "mat" are closer in the vector space.

GloVe (Global Vectors for Word Representation):

Combines the benefits of count-based and predictive models by factoring in co-occurrence statistics.

Example:

Words like "king" and "queen" are similar but differ along the gender dimension.

Fast Text:

Represents words as a combination of character n-grams, enabling the model to understand rare or out-of-vocabulary words.

Example:

Words like "walking" and "walked" are represented similarly due to shared subword components.

BERT (Bidirectional Encoder Representations from Transformers):

Generates contextual embeddings by understanding the meaning of a word in its sentence.

Example:

The word "bank" in "river bank" and "financial bank" has different embeddings based on context.



Examples of Vector Semantics

1. Word Similarity:

Words with similar meanings have closer embeddings.

Example:

"Happy" and "Joyful" will have high cosine similarity.

2. Synonyms and Analogies:

Word embeddings can identify synonyms and solve analogies.

Example:

Analogy: "Man is to King as Woman is to ?" → Answer: "Queen"

3. Document Similarity:

Entire documents can be represented as vectors (e.g., sentence or paragraph embeddings).

Example:

Comparing the similarity of two documents for plagiarism detection.

Applications of Vector Semantics and Embeddings

1. Search Engines:

- Embeddings help search engines understand synonyms and improve query results.
- Example:
 - Searching for "laptop" retrieves results for "notebook."

2. Sentiment Analysis:

- Embeddings capture the sentiment of words and sentences.
- Example:
 - Positive words ("great," "excellent") cluster together, distinct from negative words.

3. Machine Translation:

- Models like Word2Vec map words from different languages into a shared embedding space for translation.
- Example:
 - "Bonjour" (French) and "Hello" (English) are close in vector space.

1. Speech Recognition:

- Embeddings improve recognition systems by linking phonemes to meaningful words.
- Example:
 - The phrase "recognize speech" vs. "wreck a nice beach."

2. Chatbots and Virtual Assistants:

- Use embeddings to understand and respond to user queries.
- Example:
 - Recognizing "What's up?" as a casual greeting.

Advantages of Vector Semantics

1. Efficient Representation:

Reduces dimensionality compared to sparse one-hot encodings.

2. Captures Semantic Relationships:

Words with similar meanings are close in the vector space.

3. Adaptable to Various Tasks:

Supports a wide range of NLP and speech tasks.



10.ASSIGNMENT 1- UNIT 1

1. Explain the concept of Speech and Language Processing and its significance in modern applications like virtual assistants and machine translation. Provide examples to illustrate the integration of speech and text processing in real-world systems

2. Regular expressions are powerful tools for text processing. Write a detailed explanation of the following regular expressions and their practical applications:

a) `\d{4}-\d{2}-\d{2}`

b) `^[A-Za-z]+\@[A-Za-z]+\.[A-Za-z]{2,}$`

c) `\bcat\b`

For each, provide examples of text where the pattern would match or fail to match, and explain why.

3. Define Text Normalization and discuss its importance in preprocessing text for speech and language analytics. Include the following tasks in your discussion:

a) Lowercasing

b) Expanding contractions

c) Removing special characters and stop words

Task: Apply text normalization on the sentence:

"Isn't it a beautiful day for NLP? Let's clean this text!"

Provide the intermediate steps and the final output.

4. The Edit Distance measures the similarity between two strings by counting the minimum number of operations (insertion, deletion, substitution) required to transform one string into another.

Task:

a) Compute the edit distance between "intention" and "execution" using the dynamic programming approach.

b) Draw the table showing intermediate steps.

c) Discuss how edit distance can be used in applications like spell-checkers and DNA sequence analysis.

5. Discuss the role of N-gram Language Models in predicting word sequences. Explain how the Markov Assumption simplifies the modelling of sequences in N-grams.

Extension: Provide an overview of how Vector Semantics and Word Embeddings complement N-gram models by capturing the semantic relationships between words. Include examples to compare the two approaches in handling the phrase: "The dog barked loudly."



11. PART A Q & A (WITH K LEVEL AND CO) UNIT 1

1. What is the primary goal of Speech and Language Processing?(K1,CO1)

The primary goal is to enable machines to process, understand, and generate human language in textual and spoken forms.

2. What is a regular expression? (K1,CO1)

A regular expression (regex) is a sequence of characters that defines a search pattern, often used for pattern matching in strings.

3. Write a regex to find all words that start with 's' in a text.(K2,CO1)

The regex is `\bs\w*`.

4. What is text normalization? (K1,CO1)

Text normalization is the process of transforming text into a standard form, including tasks like lowercasing, removing punctuation, and expanding contractions.

5. Why is text normalization important in NLP? (K2,CO1)

It helps reduce noise in the data and ensures consistency, which improves the performance of NLP models.

6. What is edit distance? (K1,CO1)

Edit distance is the minimum number of operations (insertions, deletions, or substitutions) required to transform one string into another.

8. What is lemmatization? ?(K1,CO1)

Lemmatization reduces words to their base or root form (lemma) using vocabulary and morphological analysis, e.g., "running" → "run."

9. How does lemmatization differ from stemming? ?(K3,CO1)

Lemmatization considers the context and produces valid root forms, while stemming often removes suffixes without ensuring valid words.

10. What is stemming in NLP? ?(K1,CO1)

Stemming is the process of reducing words to their base or root form by stripping affixes, e.g., "playing" → "play."

10. Name an algorithm commonly used for stemming. ?(K1,CO1)

The Porter Stemmer algorithm.

11. What is an n-gram? ?(K1,CO1)

An n-gram is a contiguous sequence of n items (words or characters) from a given text, e.g., "the cat" is a 2-gram (bigram).

12. What is the role of n-gram models in NLP? ?(K1,CO1)

N-gram models predict the probability of a sequence of words and are used for tasks like text generation and speech recognition.

13. What are word embeddings? ?(K1,CO1)

Word embeddings are dense vector representations of words in a continuous vector space that capture their meanings and relationships.

14. Name two popular methods for creating word embeddings. ?(K1,CO1)

Word2Vec and GloVe.

12.PART B Q s (WITH K LEVEL AND CO) UNIT 1

1. Explain how regular expressions are used in text normalization tasks. Design a Python-based approach to remove punctuation, handle case normalization, and tokenize a given text input.(K4,CO1)
2. Discuss the role of text normalization in preprocessing for natural language processing (NLP). Illustrate its importance with examples.(K2,CO1)
3. What is Edit Distance, and how is it calculated? Derive the dynamic programming algorithm for computing Levenshtein Distance between two strings. Provide an example.(K1,K4,CO1)
4. Explain the significance of Edit Distance in NLP applications like spell correction and plagiarism detection. Solve a problem where the Edit Distance between two strings is computed step by step.(K4,C01)
5. Differentiate between stemming and lemmatization. Discuss their advantages and disadvantages with examples. How do these methods affect downstream tasks in NLP?(K3,CO1)
6. Given a list of words, perform stemming and lemmatization using Python (use any library like NLTK or spaCy). Discuss the results for each word.(K1,CO1)
7. Explain the working of N-gram language models. Derive the formula for calculating probabilities using a bigram model with an example dataset.(K4,CO1)
8. Discuss the limitations of N-gram language models compared to neural network-based language models. Provide examples to support your answer.(K2,CO1)

9. What are word embeddings? Compare Word2Vec, GloVe, and FastText in terms of their methodology and applications.(K1,CO1)

10. Discuss the role of vector semantics in capturing contextual and semantic relationships between words. Illustrate with an example using a pre-trained embedding (e.g., GloVe).(K2,CO1)



13. SUPPORTIVE ONLINE CERTIFICATION COURSES UNIT 1

1. Nptel Course Link:

[Natural Language Processing - Course](#)

2. Coursera:

[Text Mining and Analytics | Coursera](#)

[Hands-on Text Mining and Analytics | Coursera](#)

3. Udemy

[Applied Natural Language Processing - Course](#)

4. edX:

[Applied Natural Language Processing - Course](#)

14. REAL TIME APPLICATIONS IN DAY-TO-DAY LIFE AND TO INDUSTRY

1. Voice Assistants and Smart Devices

- **Examples:** Alexa, Siri, Google Assistant.
- **Functionality:** Speech-to-text conversion for understanding commands, text-to-speech for responses, contextual understanding for task execution.

2. Chatbots and Virtual Assistants

- **Examples:** Customer support bots (e.g., in banking, e-commerce).
- **Functionality:** Natural language understanding (NLU) for query resolution and conversation flow.

3. Social Media Monitoring

- **Examples:** Sentiment analysis on platforms like Twitter, Instagram, and Facebook.
- **Functionality:** Detecting public sentiment, monitoring trends, and identifying brand mentions.

4. Spelling and Grammar Checking

- **Examples:** Grammarly, Microsoft Word suggestions.
- **Functionality:** Real-time text analysis for grammar correction and style improvements using NLP.

5. Content Recommendation Systems

- **Examples:** Netflix subtitles, YouTube captions, Spotify song lyrics.
- **Functionality:** Generating and analyzing text for improved user experience.

6. Language Translation

- **Examples:** Google Translate, Microsoft Translator.
- **Functionality:** Real-time multilingual communication with speech and text translation.

Assessment Schedule

(Proposed Date & Actual Date)

Sl. No.	ASSESSMENT	Proposed Date	Actual Date
1	FIRST INTERNAL ASSESSMENT		
2	SECOND INTERNAL ASSESSMENT		
3	MODEL EXAMINATION		
4	END SEMESTER EXAMINATION		

16.PRESCRIBED TEXT BOOKS & REFERENCE BOOKS

TEXT BOOKS:

1. Jurafsky, D. and J. H. Martin, Speech and language processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition Pearson Publication, Third Edition, 2022.
2. Lawrence Rabiner, Biing-Hwang Juang and B.Yegnanarayana, "Fundamentals of Speech Recognition", Pearson Education, 2009.

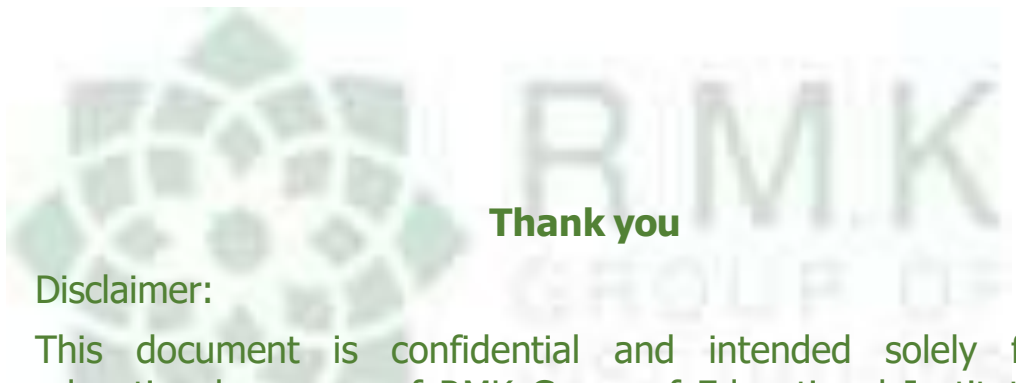
REFERENCES:

1. John Atkinson-Abutridy, Text Analytics: An Introduction to the Science and Applications of Unstructured Information Analysis, CRC Press, 2022.
2. Jim Schwoebel, NeuroLex, Introduction to Voice Computing in Python, 2018
3. Lawrence R. Rabiner, Ronald W. Schafe, Theory and Applications of Digital Speech Processing, First Edition, Pearson, 2010.



17.MINI PROJECT SUGGESTIONS

- 1.Implement a basic voice cloning system to mimic a target speaker's voice using open-source TTS libraries.
2. Design a chatbot using predefined rules for FAQs in a specific domain (e.g., healthcare or banking)
3. Develop a question-answering model using pre-trained transformers (e.g., BERT-based QA) on a dataset like SQuAD.
4. Build a sentiment classifier using traditional ML methods (e.g., Naïve Bayes, SVM) or neural networks on datasets like IMDB reviews or Twitter data.
5. Train a text classification model to categorize news articles into predefined topics using transformers (e.g., BERT).



Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.